



ALGORITMO Y ESTRUCTURA DE DATOS BASADOS EN INTELIGENCIA ARTIFICIAL_2610

PA 1 – CASO INTEGRADOR: CAMPUSNAVIGATOR IA



APELLIDOS Y NOMBRES	CORREO ELECTRÓNICO	PARTICIPACIÓN
Velásquez Fuero Nicolý Dayana	72779722@mail.isil.pe	100%
Salas Tucto Nadia Salisa	75694547@mail.isil.pe	100%
Bedon Saboya Carlos Josué	73805189@mail.isil.pe	100%

PERIODO 2026 – I

DOCENTE

WILDER JULIO ESPINOZA BRAVO

ÍNDICE

Tabla de contenido

1. Actividad 1. Recursividad y trazabilidad del algoritmo	2
1.1. ¿Qué caracteriza a un algoritmo recursivo?	2
1.2. Relación con el principio divide y vencerás	2
1.3. Diseño de algoritmo recursivo: Conteo de pasos de revisión de puntos de control	3
1.3.1. Definición matemática del problema	3
1.3.2. Pseudocódigo y explicación de cada proceso	3
1.3.3. Traza manual de ejecución (caso de prueba: $n = 5$)	3
1.3.4. Caso base, caso recursivo y criterio de finalización	4
1.3.5. Caso de aplicación (NetBeans)	5
2. Actividad 2. Ordenamiento recursivo aplicado	5
2.1. Vector de tiempos de desplazamiento	5
2.2. Pseudocódigo del Merge Sort y explicación por fases	6
2.3. Traza de la división y combinación del vector	7
2.4. Análisis de eficiencia	7
2.5. Implementación (NetBeans)	8
3. Actividad 3. Modelado del problema con grafos y búsquedas	9
3.1. Modelado del campus como grafo	9
3.2. Recorrido DFS y BFS	10
3.3. Caso práctico para a^* (búsqueda heurística)	11
3.4. Comparación de búsquedas en tres escenarios	12
4. Actividad 4: Sustento técnico y cierre grupal	13
4.1. Conclusión grupal: aprendizajes al combinar recursividad y grafos	13
4.2. Pertinencia de la solución y mejoras para una segunda versión	13
4.3. Organización de la entrega en GitHub	14
5. Video	14

Una institución educativa desea desarrollar un módulo llamado CampusNavigator IA para orientar a estudiantes dentro del campus. El sistema debe resolver tres necesidades: (1) ordenar resultados y recorridos mediante una estrategia recursiva, (2) modelar rutas entre ambientes, laboratorios y servicios usando grafos, y (3) recomendar recorridos o búsquedas adecuadas según el objetivo del usuario. El equipo deberá proponer una solución académica que evidencie análisis, diseño algorítmico y justificación técnica.

1. Actividad 1. Recursividad y trazabilidad del algoritmo

1.1. ¿Qué caracteriza a un algoritmo recursivo?

Un algoritmo recursivo es aquel que se define en términos de sí mismo: para resolver un problema, la función se invoca a sí misma sobre una versión más pequeña del mismo problema. Para que la recursión funcione correctamente y no se produzca un desbordamiento de pila, todo algoritmo recursivo debe cumplir dos condiciones fundamentales:

- Caso base: es la condición que detiene la recursión. Representa el problema más pequeño o trivial, que se resuelve de forma directa sin volver a llamar a la función.
- Caso recursivo: es la llamada de la función a sí misma con un subproblema de menor tamaño, acercándose progresivamente al caso base.

1.2. Relación con el principio divide y vencerás

La recursividad es la herramienta natural para implementar el principio divide y vencerás, que consiste en tres pasos: dividir el problema en subproblemas más pequeños de la misma naturaleza, vencer (resolver) esos subproblemas de forma recursiva, y combinar los resultados parciales para construir la solución global. La relación es directa: la fase de división genera nuevas llamadas recursivas, la fase de vencer termina cuando se alcanza el caso base, y la fase de combinación reúne los resultados a medida que la pila de llamadas se desapila. En CampusNavigator IA, este principio permite contar pasos de revisión y, más adelante, ordenar tiempos de desplazamiento mediante Merge Sort.

1.3. Diseño de algoritmo recursivo: Conteo de pasos de revisión de puntos de control

Se diseña un algoritmo recursivo que calcula el número total de pasos necesarios para revisar una secuencia de puntos de control del campus. Cada punto de control revisado equivale a un paso. El problema de revisar n puntos se reduce a: revisar el primer punto (1 paso) más revisar los $(n-1)$ puntos restantes.

1.3.1. Definición matemática del problema

Si $\text{contarPasos}(n)$ es el total de pasos para revisar n puntos de control, entonces se cumple la relación de recurrencia:

$$\begin{aligned}\text{contarPasos}(0) &= 0 && \text{(caso base)} \\ \text{contarPasos}(n) &= 1 + \text{contarPasos}(n - 1) && \text{(caso recursivo)}\end{aligned}$$

1.3.2. Pseudocódigo y explicación de cada proceso

```
FUNCIÓN contarPasos(n):  
    // n = cantidad de puntos de control por revisar  
    SI n == 0 ENTONCES          // caso base  
        RETORNAR 0             // no quedan puntos, 0 pasos  
    FIN SI  
    // caso recursivo: 1 paso por el punto actual  
    // + los pasos de los (n-1) puntos restantes  
    RETORNAR 1 + contarPasos(n - 1)  
FIN FUNCIÓN
```

Explicación de cada proceso:

- (1) La función recibe n , el número de puntos de control pendientes.
- (2) Se evalúa el caso base: si $n = 0$ no queda nada por revisar y se retorna 0, deteniendo la recursión.
- (3) Si $n > 0$, se cuenta 1 paso por revisar el punto actual y se suma el resultado de la llamada recursiva $\text{contarPasos}(n-1)$, que resuelve el mismo problema con un punto menos.
- (4) Al desapilarse la pila, las sumas parciales se combinan hasta obtener el total.

1.3.3. Traza manual de ejecución (caso de prueba: $n = 5$)

Proponemos como caso de prueba $n = 5$ puntos de control. La traza descendente muestra las llamadas hasta el caso base, y la ascendente muestra la combinación de resultados. Se producen 6 llamadas (niveles 0 a 5).

Nivel	Llamada	¿Caso base?	Expresión evaluada	Valor retornado
0	contarPasos(5)	No	$1 + \text{contarPasos}(4)$	5
1	contarPasos(4)	No	$1 + \text{contarPasos}(3)$	4
2	contarPasos(3)	No	$1 + \text{contarPasos}(2)$	3
3	contarPasos(2)	No	$1 + \text{contarPasos}(1)$	2
4	contarPasos(1)	No	$1 + \text{contarPasos}(0)$	1
5	contarPasos(0)	Sí	retorna 0	0

Combinación ascendente de resultados: $0 \rightarrow 1+0=1 \rightarrow 1+1=2 \rightarrow 1+2=3 \rightarrow 1+3=4 \rightarrow 1+4=5$. El algoritmo retorna 5, que coincide con el número de puntos de control revisados. El resultado es correcto.

1.3.4. Caso base, caso recursivo y criterio de finalización

Elemento	Definición en el algoritmo
Caso base	$n = 0$: no quedan puntos por revisar, retorna 0 y detiene la recursión.
Caso recursivo	$n > 0$: retorna $1 + \text{contarPasos}(n - 1)$, reduciendo el problema en una unidad.
Criterio de finalización	En cada llamada n disminuye en 1; como parte de un valor finito y decreciente, se garantiza que llega a 0 en un número finito de pasos, evitando recursión infinita.

1.3.5. Caso de aplicación (NetBeans)

```
package campusnavigator+ia;

public class ConteoPuntosControl {

    // Algoritmo recursivo: cuenta los pasos de revisión
    public static int contarPasos(int n) {
        if (n == 0) {                // caso base
            return 0;
        }
        // caso recursivo: 1 paso + los (n-1) restantes
        return 1 + contarPasos(n - 1);
    }

    public static void main(String[] args) {
        int puntos = 5;              // caso de prueba
        int total = contarPasos(puntos);
        System.out.println("Puntos de control: " + puntos);
        System.out.println("Total de pasos de revision: " + total);
    }
}
```

Salida esperada en la consola de NetBeans: “Puntos de control: 5” y “Total de pasos de revision: 5”, confirmando la traza manual.

2. Actividad 2. Ordenamiento recursivo aplicado

2.1. Vector de tiempos de desplazamiento

Se considera un conjunto de tiempos estimados de desplazamiento (en minutos) entre puntos del campus. Proponemos el siguiente vector de 8 valores, que corresponden a los tiempos entre pares de ambientes registrados por CampusNavigator IA:

tiempos = [8, 3, 5, 2, 9, 1, 4, 7]

El objetivo es organizar estos tiempos de menor a mayor para que el sistema muestre primero las rutas más rápidas. Se elige el algoritmo Merge Sort, un algoritmo recursivo de ordenamiento basado en el principio divide y vencerás, estable y con complejidad garantizada $O(n \log n)$.

2.2. Pseudocódigo del Merge Sort y explicación por fases

Merge Sort resuelve el ordenamiento en tres fases: dividir el vector en dos mitades, ordenar recursivamente cada mitad y combinar (fusionar) las mitades ya ordenadas en un único vector ordenado.

```
FUNCIÓN mergeSort(A, inicio, fin):  
    SI inicio < fin ENTONCES          // hay 2 o más elementos  
        medio = (inicio + fin) / 2 // DIVIDIR  
        mergeSort(A, inicio, medio) // VENCER izquierda  
        mergeSort(A, medio + 1, fin) // VENCER derecha  
        combinar(A, inicio, medio, fin) // COMBINAR  
    FIN SI  
FIN FUNCIÓN  
  
FUNCIÓN combinar(A, inicio, medio, fin):  
    crear listas L = A[inicio..medio], R = A[medio+1..fin]  
    i = 0; j = 0; k = inicio  
    MIENTRAS i < tam(L) Y j < tam(R):  
        SI L[i] <= R[j] ENTONCES A[k] = L[i]; i++  
        SINO A[k] = R[j]; j++  
        k++  
    copiar elementos restantes de L y luego de R en A  
FIN FUNCIÓN
```

Explicación de cada proceso

- Dividir: se calcula el índice medio y el vector se parte en dos mitades. La división continúa recursivamente hasta que cada subvector tiene un solo elemento (caso base implícito: inicio == fin), que por definición ya está ordenado.

- Vencer: cada mitad se ordena mediante una llamada recursiva a mergeSort. Es aquí donde el problema grande se transforma en subproblemas idénticos, pero más pequeños.
- Combinar: la función combinar recorre simultáneamente las dos mitades ordenadas y las fusiona comparando elemento a elemento, produciendo un tramo ordenado. Al combinarse todos los tramos, el vector completo queda ordenado.

2.3. Traza de la división y combinación del vector

Aplicando Merge Sort al vector [8, 3, 5, 2, 9, 1, 4, 7], la fase de división genera el siguiente árbol de recursión, y la fase de combinación reconstruye el vector ordenado de abajo hacia arriba:

Fase	Detalle del proceso
División nivel 1	[8,3,5,2,9,1,4,7] → [8,3,5,2] [9,1,4,7]
División nivel 2	[8,3] [5,2] [9,1] [4,7]
División nivel 3	[8][3] [5][2] [9][1] [4][7] (subvectores de 1 elemento)
Combinación nivel 3	[3,8] [2,5] [1,9] [4,7]
Combinación nivel 2	[2,3,5,8] [1,4,7,9]
Combinación nivel 1	[1,2,3,4,5,7,8,9] ← vector ordenado final

El vector queda ordenado de menor a mayor: [1, 2, 3, 4, 5, 7, 8, 9]. La estrategia recursiva es válida porque cada subproblema (ordenar una mitad) es idéntico al problema original pero de menor tamaño, y el caso base (un solo elemento) siempre se alcanza, garantizando la finalización.

2.4. Análisis de eficiencia

Devuelve el elemento al frente de la cola sin eliminarlo.

Aspecto	Merge Sort
Mejor caso	$O(n \log n)$
Caso promedio	$O(n \log n)$
Peor caso	$O(n \log n)$

Aspecto	Merge Sort
Complejidad espacial	$O(n)$ por los subvectores auxiliares en la fusión
Estabilidad	Estable (conserva el orden de elementos iguales)

La eficiencia se explica por la estructura del algoritmo: el vector se divide $\log_2(n)$ veces (altura del árbol de recursión) y en cada nivel se realiza un trabajo de combinación proporcional a n . El producto $n \times \log n$ se mantiene en todos los escenarios, lo que hace a Merge Sort predecible y muy adecuado para ordenar los tiempos de desplazamiento de CampusNavigator IA, incluso si el número de rutas crece de forma considerable.

2.5. Implementación (NetBeans)

```
package campusnavigator+ia;

public class OrdenarTiempos {

    public static void mergeSort(int[] A, int inicio, int fin) {
        if (inicio < fin) {
            int medio = (inicio + fin) / 2;    // DIVIDIR
            mergeSort(A, inicio, medio);      // VENCER izq.
            mergeSort(A, medio + 1, fin);     // VENCER der.
            combinar(A, inicio, medio, fin);   // COMBINAR
        }
    }

    private static void combinar(int[] A, int ini, int med, int fin) {
        int n1 = med - ini + 1;
        int n2 = fin - med;
        int[] L = new int[n1];
        int[] R = new int[n2];
        for (int i = 0; i < n1; i++) L[i] = A[ini + i];
        for (int j = 0; j < n2; j++) R[j] = A[med + 1 + j];
        int i = 0, j = 0, k = ini;
        while (i < n1 && j < n2) {
```

```

        if (L[i] <= R[j]) A[k++] = L[i++];
        else             A[k++] = R[j++];
    }
    while (i < n1) A[k++] = L[i++];
    while (j < n2) A[k++] = R[j++];
}

public static void main(String[] args) {
    int[] tiempos = {8, 3, 5, 2, 9, 1, 4, 7};
    mergeSort(tiempos, 0, tiempos.length - 1);
    System.out.print("Tiempos ordenados: ");
    for (int x : tiempos) System.out.print(x + " ");
}
}

```

Salida esperada en NetBeans: “Tiempos ordenados: 1 2 3 4 5 7 8 9”.

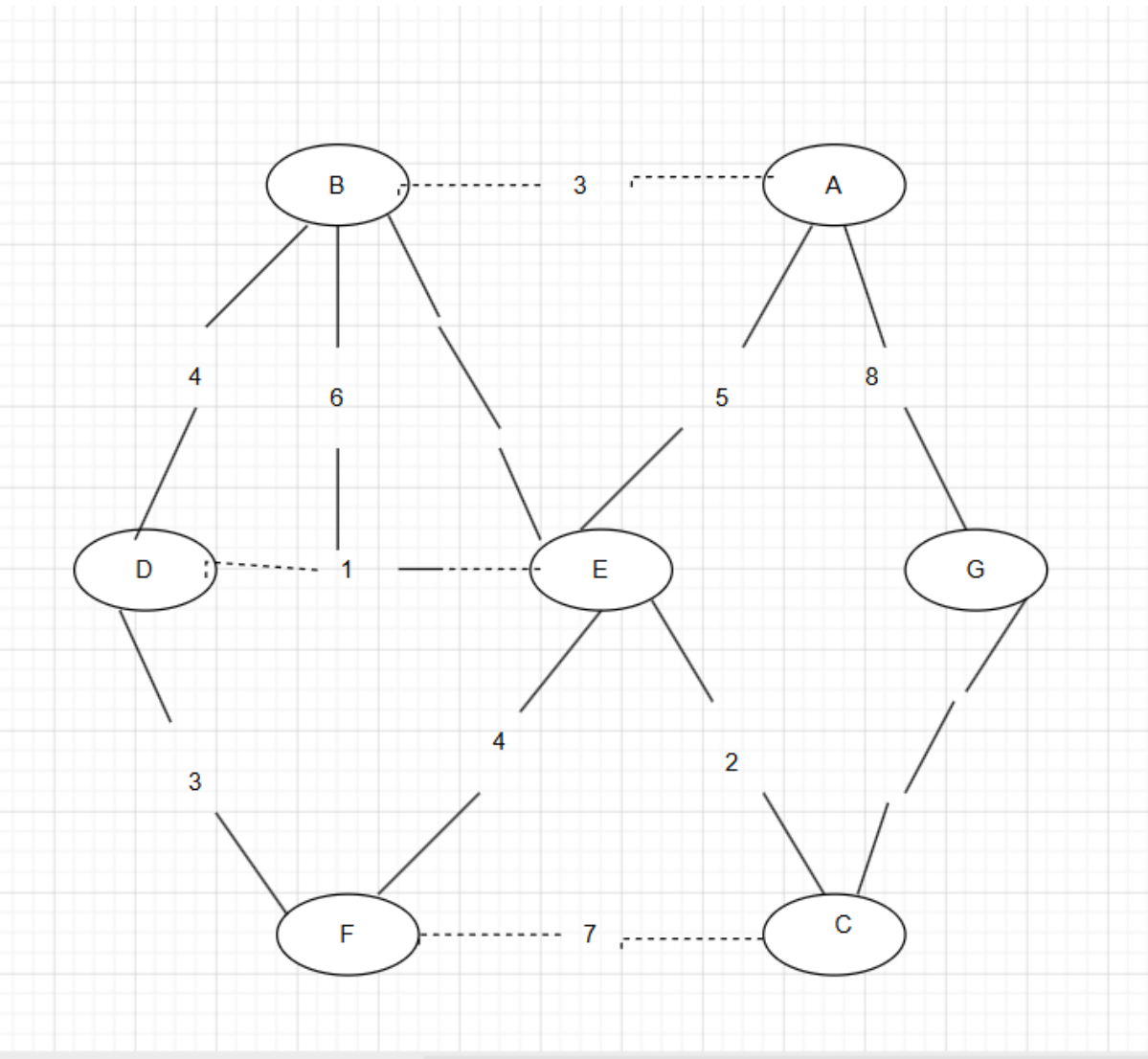
3. Actividad 3. Modelado del problema con grafos y búsquedas

3.1. Modelado del campus como grafo

Código	Lugar
A	Biblioteca
B	auditoria principal
C	Cafetería
D	Laboratorio de computación
E	Oficina de bienestar
F	Aula magna
G	Estacionamiento

Origen	Destino	Tiempo(min)
A	B	3
A	C	5
A	G	8

B	D	4
B	E	6
C	F	7
C	G	2
D	E	1
D	F	3
E	F	4



3.2. Recorrido DFS y BFS

RECORRIDO DFS desde nodo A

A → B → D → E → F → C → G

Explicación:

1. Inicio en A → visitado
2. Vecinos de A: B, C, G → elige B (alfabético)
3. Vecinos de B: A, D, E → A ya visitado, elige D
4. Vecinos de D: B, E, F → B visitado, elige E
5. Vecinos de E: B, D, F → B y D visitados, elige F
6. Vecinos de F: C, D, E → D y E visitados, elige C
7. Vecinos de C: A, F, G → A y F visitados, elige G
8. Vecinos de G: A, C → ambos visitados → FIN

RECORRIDO BFS desde nodo A

A → B → C → G → D → E → F

Explicación :

1. Cola: [A] → visitar A, encolar B, C, G
2. Cola: [B, C, G] → visitar B, encolar D, E
3. Cola: [C, G, D, E] → visitar C, encolar F (A y G ya visitados)
4. Cola: [G, D, E, F] → visitar G (sin nuevos vecinos)
5. Cola: [D, E, F] → visitar D (sin nuevos vecinos no visitados)
6. Cola: [E, F] → visitar E (F ya encolado)
7. Cola: [F] → visitar F → FIN

3.3. Caso práctico para a* (búsqueda heurística)

Escenario:

Un estudiante está en la **Biblioteca (A)** y quiere llegar al **Estacionamiento (G)** en el menor tiempo posible, pero hay tráfico variable. Usaremos A* para encontrar la ruta óptima.

Heurística propuesta (h(n)):

Distancia en línea recta estimada (en minutos) desde cada nodo hasta G:

Nodo	h(n) (tiempo estimado a G)
A	8

B	10
C	2
D	6
E	5
F	4
G	0

Cálculo A ($f(n) = g(n) + h(n)$):*

Paso	Nodo	g(n) (costo real desde A)	h(n)	f(n)	Elegido
1	A	0	8	8	A
2	B	3	10	13	-
2	C	5	2	7	C
2	G	8	0	8	-
3	F(desde C)	5+7=12	4	16	-
3	G(desde C)	5+2=7	0	7	G

Ruta encontrada por A*: A → C → G (costo total = 7 minutos)

Ruta óptima real: A → C → G = 7 minutos (coincide con A*)

3.4. Comparación de búsquedas en tres escenarios

ESCENARIO	Búsqueda recomendada	Justificación técnica
Exploración completa del campus (ej: inventario de equipos)	DFS	Recorre todos los nodos sin importar optimalidad. Útil para mapeo completo o backup de datos.
Búsqueda de proximidad (ej: encontrar el nodo más cercano a uno dado)	BFS	Encuentra el camino con menos aristas. Ideal para saber qué lugares están más cerca en cantidad de conexiones.
Ruta recomendada con criterio heurístico (ej: navegación con tiempos)	A*	Minimiza costo real + heurístico. Encuentra ruta óptima en grafos ponderados. Mejor para recomendación de rutas.

4. Actividad 4: Sustento técnico y cierre grupal

4.1. Conclusión grupal: aprendizajes al combinar recursividad y grafos

Al combinar recursividad y grafos para resolver el caso CampusNavigator IA, el grupo comprendió que ambos enfoques se complementan: la recursividad y el principio divide y vencerás nos permitieron descomponer problemas grandes (como ordenar todos los tiempos de desplazamiento) en subproblemas idénticos más pequeños, obteniendo soluciones elegantes y eficientes como Merge Sort con complejidad $O(n \log n)$. Por su parte, la teoría de grafos nos dio el lenguaje para modelar la realidad del campus —ambientes como nodos y caminos como aristas ponderadas— y los recorridos DFS, BFS y A* nos dieron herramientas para explorarlo y recomendar rutas según el objetivo del usuario. Aprendimos que elegir la estructura y el algoritmo correctos no es un detalle menor: determina la eficiencia, la memoria utilizada y la calidad de la solución.

4.2. Pertinencia de la solución y mejoras para una segunda versión

La solución es pertinente para un entorno educativo porque responde a una necesidad real —orientar a los estudiantes dentro del campus— con criterios técnicos sólidos y verificables. El ordenamiento recursivo permite mostrar primero las rutas más rápidas; el modelado con grafos refleja fielmente la conectividad física del campus; y la búsqueda A* recomienda recorridos óptimos hacia un destino, ahorrando tiempo al estudiante.

Como mejoras para una segunda versión, el grupo propone:

- Incorporar pesos dinámicos en las aristas que consideren congestión por horarios de clase o rutas de accesibilidad (rampas, ascensores).
- Añadir una capa de IA que aprenda de los recorridos frecuentes y personalice las recomendaciones para cada estudiante.
- Extender el grafo a varios pisos y edificios usando un grafo por niveles, e integrar geolocalización en tiempo real.

- Optimizar A* con estructuras de datos más eficientes (cola de prioridad / heap) para escalar a campus de mayor tamaño.

4.3. Organización de la entrega en GitHub

La entrega técnica se organiza en un repositorio de GitHub con una estructura de carpetas clara que separa la documentación, el código fuente y las evidencias:

```
CampusNavigator-IA/
|
|-- /docs      -> Documento PA4 (PDF) y diagramas del grafo
|-- /src       -> Código Java (NetBeans):
|               ConteoPuntosControl.java
|               OrdenarTiempos.java
|               BusquedasCampus.java
|-- /evidencias -> Capturas de ejecución y traza de pruebas
|-- README.md  -> Descripción del trabajo y enlace del video
```

Contenido del README.md

Sección del README	Contenido
Título del trabajo	PA4 – CampusNavigator IA: Recursividad y Grafos.
Integrantes	Nombres completos y correos del grupo.
Propósito	Breve explicación de la solución y objetivo del módulo.
Instrucciones de ejecución	Cómo abrir y ejecutar cada clase Java en NetBeans.
Conclusiones del grupo	Síntesis de los aprendizajes (Actividad 4).
Enlace del video	URL de YouTube (No listado) con la sustentación grupal.

5. Video

 Enlace video Youtube: